

Bases de données

Fiche n°1

Notion de Bases de données

Une donnée : est une information qui peut effectuer des traitements	Une information : c'est une donnée regroupée avec d'autres liées à un contexte donnée.	Entité : c'est la représentation d'un objet du monde réel ayant une existence propre. (Exp : Etudiant, Livre, classe...)
Propriété (attribut) : représente les caractéristiques les propriétés de l'entité.	Association : c'est un outil permettant de représenter les liens entre les entités.	Base de Données : est un ensemble de données non redondantes, structurées et persistantes (non volatiles) relatives à un ou plusieurs domaines du monde réel.
Intérêts de l'utilisation d'une BD : -Centralisation -Indépendance entre données et programme -Intégrité de données -Partage de données (concurrence d'accès)	Intégrité de données : Ensemble de règles appelées contraintes d'intégrité qui décrivent les propriétés qui doivent être vérifiées par les données pour être enregistrées. Elle permet d'assurer la cohérence des données.	Modèles de BD : -Hiérarchique -Réseau - <u>Modèle Relationnel</u> : le principe consiste à représenter aussi bien les objets que les liens à l'aide d'une structure appelée table .
Système de Gestion de Base de Données(SGBD) : c'est un logiciel qui permet de créer, de gérer et d'interroger efficacement une BD. (Pour dialoguer avec le SGBD il faut utiliser le langage SQL (Structured Query Language))	Les fonctions de bases d'un SGBD : -Définition des données -Manipulation des données -Intégrité des données -la gestion des accès concurrents -la confidentialité -la sécurité de fonctionnement	Table : une table est un ensemble de données relatives à un même sujet(ou entité) et structurées sous forme de tableau. Colonne (ou champ ou propriété) : Dans une table une colonne est une propriété élémentaire de l'objet décrit par cette table. Ligne (n-uplet ou enregistrement) : Une ligne correspond à un objet représenté par la table.
Clé primaire (identifiant=primary key) : c'est un identificateur unique pour la table qui correspond à une colonne ou un ensemble de colonnes.	Clé étrangère =référence =contrainte d'intégrité référentielle = foreign key Un lien entre deux tables A et B est représenté par l'ajout dans la table B d'une nouvelle colonne correspondant à la clé primaire de la table A. cette nouvelle colonne est appelée Clé étrangère .	Contraintes d'intégrité : Ensemble de règles appliquées à une colonne ou une table et qui doit être toujours vérifiée. Les principales contraintes d'intégrité sont : Contraintes de domaines : Ce sont des contraintes appliquées sur les colonnes. Elles permettent de fixer caractère obligatoire ou non d'une colonne et les règles de validité des valeurs qui peuvent être prise par cette colonne. Contraintes d'intégrité de table : Elles permettent d'assurer que chaque table a une clé primaire. Contrainte d'intégrité référentielle : Elles permettent de s'assurer que les valeurs introduites dans une colonne se figurent dans une autre comme clé primaire.

Exemple: table Elève

Colonnes				Lignes
Clé primaire	Numéro	Nom	Prénom	
	1	Saadallah	Kais	18
	2	Tounsi	Leila	22
	3	khayati	wissem	19

****Exemples pour les relations ****

Relation un à plusieurs

Un élève appartient à une seule classe. Dans une classe on trouve plusieurs élèves

ELEVE ∞

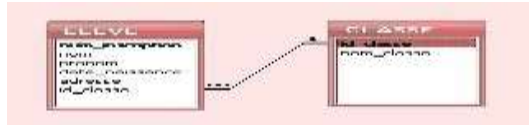
1 CLASSE

Donc CLASSE table mère et ELEVE table fille. Comme CLASSE est mère alors sa clé primaire migre à la table fille qui est ELEVE et sera une clé étrangère

Description textuelle :

ELEVE (num_inscription, nom, prenom, date_naissance, adresse, id_classe#)

CLASSE(id_classe, nom_classe)

Description graphique :**Relation plusieurs à plusieurs**

Un athlète peut participer à plusieurs courses. Dans une course on trouve plusieurs athlètes. Dans ce cas la relation participer se transforme en un table identifié par deux clés primaires qui sont les clés primaires de la table athlète et la table Courses(les deux clés seront primaires et étrangères au même temps dans la nouvelle table)

Description textuelle :

athlete (anum, anom, aage)

participation (cnum#, anum #)

course (cnum, cnom)

Description graphique :**Fiche n°2****Création et modification de la structure d'une base de données**

Le mode commande consiste à créer les différentes structures de la base de données à l'aide de commandes du langage **SQL (Structured Query Language)**

SQL est un langage informatique utilisé pour exploiter des bases de données. Il permet de façon générale la définition, la manipulation et le contrôle de sécurité de données.

Dans la pratique, le langage **SQL** est utilisé pour créer des tables, ajouter des enregistrements sous forme de lignes, interroger une base de données, la mettre à jour, ou encore gérer les droits d'utilisateurs de cette base de données.

Ce langage est composé de trois familles de commandes:

1. Langage de définition de données (LDD)
 - création de tables : **CREATE TABLE**
 - modification de tables: **ALTER TABLE**
 - suppression de tables: **DROP TABLE**
2. Langage de manipulation de données (LMD /DML)
 - insertion de lignes: **INSERT**
 - mise à jour des lignes: **UPDATE**
 - suppression de ligne: **DELETE**
 - Langage de requêtes : **SELECT FROM..... WHERE.....**
3. Langage de contrôle de données: GRANT, REVOKE

Soit la description textuelle de la base de données d'une agence de location de voitures:

Client (NCIN Cli, Nom_Cli, Prenom_Cli, Adresse_Cli, Ville)

Vehicule (Immat vehicule, Marque, Model, Annee_Acq)

Location (Immat Vehicule#, NCIN#, Date_Loc, Duree_Loc, Cout_Loc)

I. Langage de définition de données (LDD)**I.1 Création de la BD:****Les étapes à suivre:**

1. Taper la commande "CREATE DATABASE Gestion_
2. Pour utiliser la base déjà créée, taper la commande "USE Gestion_Location;"

Remarque:

Les commandes "CREATE DATABASE <nom_de_la_base>" et "USE <nom_BD>" sont des commandes SQL propres au SGBD MySQL et non des commandes normalisées comme celle qu'on va les voir à la suite de ce cours.

Il s'agit de même de trois commandes:

- "SHOW DATABASES" pour afficher les noms des bases de données existantes
- "SHOW TABLES" pour afficher les tables d'une base de données déjà utilisée
- "DESCRIBE <nom_table>" affiche pour la table "nom_table" ses colonnes ainsi que leurs types et contraintes de domaine s'il existe.

I.2 Création d'une table

La commande SQL qui permet de créer une table en donnant son nom, ses attributs et ses contraintes est:

```
CREATE TABLE nom_table (
  Nom_Colonne1 type_colonne1 [[CONSTRAINT] contrainte-col]*
  ...
  [[CONSTRAINT] contrainte-table);
```

Légende:

[]:option *: applicable autant de fois que souhaité mots en capitale : mots-clé.

Noter Bien:**❖ Contrainte-col : contrainte sur une colonne**

- **NOT NULL** : obligatoire
- **UNIQUE** : pas deux lignes avec la même valeur
- **PRIMARY KEY** : identifiant de la table (UNIQUE + NOT NULL)
- **REFERENCES** nom_table (nom-col)
- **CHECK (condition)**: Condition que chaque ligne de la table doit vérifier
- **AUTO_INCREMENT** : s'applique pour une colonne de type entier (identique au type Numéro_Auto en Ms. Access)

❖ Contrainte-table : contraintes sur une table

- **PRIMARY KEY** (nom-col1, [nom-col2,...])
- **FOREIGN KEY** (nom-col)* **REFERENCES** nom_table [(nom-col)*]
- **CHECK** (condition)

❖ Les principaux types de données sous MySQL**✓ Les champs numériques :**

TINYINT	Entier très petit compris entre -128 et 127 Si l'option UNSIGNED est utilisée, ce nombre sera compris entre 0 et 255
INT	Entier standard compris entre -2 147 483 648 et 2 147 483 647. Si l'option UNSIGNED est utilisée, ce nombre sera compris entre 0 et 4 294 967 295
NUMERIC (n)	Décimal de double précision
DECIMAL (entier, décimal)	Réel, définissez la longueur de chacune des deux parties.

✓ Les champs chaînes de caractères :

CHAR (n)	Chaîne de n caractère, taille fixe
VARCHAR(M)	Chaîne de caractères variable. M peut être compris entre 1 et 255.(avec 255 caractères maximum)
BLOB, TEXT	Zone de texte standard Objet d'une longueur maximale de 65535 caractères, TEXT aura un contenu de type ASCII (casse insensible) et BLOB aura un contenu de type binaire (casse sensible).(avec 65 535 caractères maximum)

✓ **Les champs de type date et heure :**

DATE	Date (ex: 2025-11-04)
TIME	Heure (ex: 23:44:05)
DATETIME	Date et heure (ex: 2025-11-04 23:44:05)
YEAR	Année (ex: 2025)

Remarque:

❶ Qu'est ce qui se passe quand on détruit une clé primaire qui est référencée par un tuple (foreign key) d'une autre table?

Pour supprimer aussi la ligne de la table fille qui correspond à la clé primaire de la table mère, il faut utiliser l'action "**ON DELETE CASCADE**" lors de la définition de clé étrangère.

❷ Lors de la création des tables en SQL, on doit toujours commencer par les tables mères.

Activité :

Créer les trois tables **Client**, **Vehicule** et **location**

Les commandes SQL permettant la création de 3 tables sont:

❖ **Table Client**

```
CREATE TABLE Client(
NCIN_Cli VARCHAR(8) PRIMARY KEY,
Nom_Cli VARCHAR(20) NOT NULL,
Prenom_Cli VARCHAR(20) NOT NULL,
Adresse_Cli VARCHAR(50),
Ville VARCHAR(20));
```

❖ **Table Véhicule**

```
CREATE TABLE Vehicule(
immat_Vehicule VARCHAR(10) primary key,
marque VARCHAR(20) NOT NULL CHECK(marque
in('Peugeot','Renault','Fiat','Opel','Volkswagen','Citroën')),
model VARCHAR(20) NOT NULL,
date_acq YEAR NOT NULL);
```

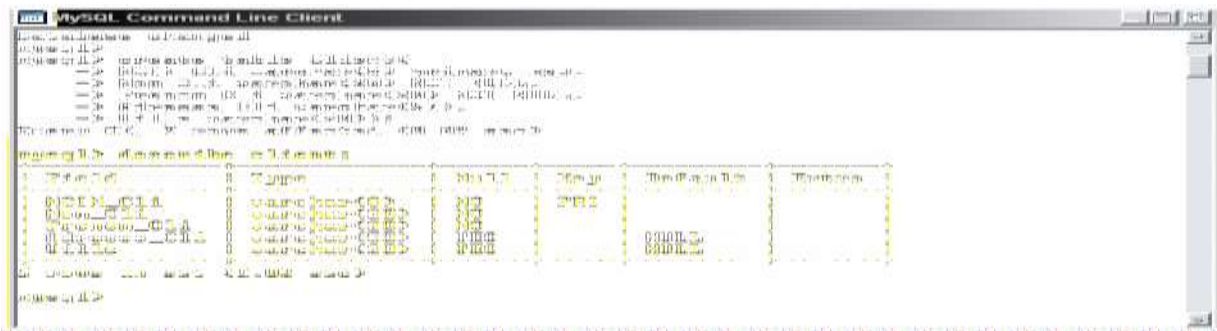
❖ **Table Location**

```
CREATE TABLE Location(
immat_Vehicule VARCHAR(10) REFERENCES vehicule(immat_Vehicule )ON DELETE
CASCADE,
NCIN_Cli VARCHAR(8) REFERENCES Client(NCIN_Cli )ON DELETE CASCADE,
date_loc TIMESTAMP DEFAULT CURRENT_TIMESTAMP C HECK(date_loc<=NOW()),
duree_loc INT(3) CHECK(duree_loc>0),
cout_loc DECIMAL(8,3) CHECK(cout_loc>0),
CONSTRAINT PK Location PRIMARY KEY(immat_Vehicule,NCIN_Cli));
```

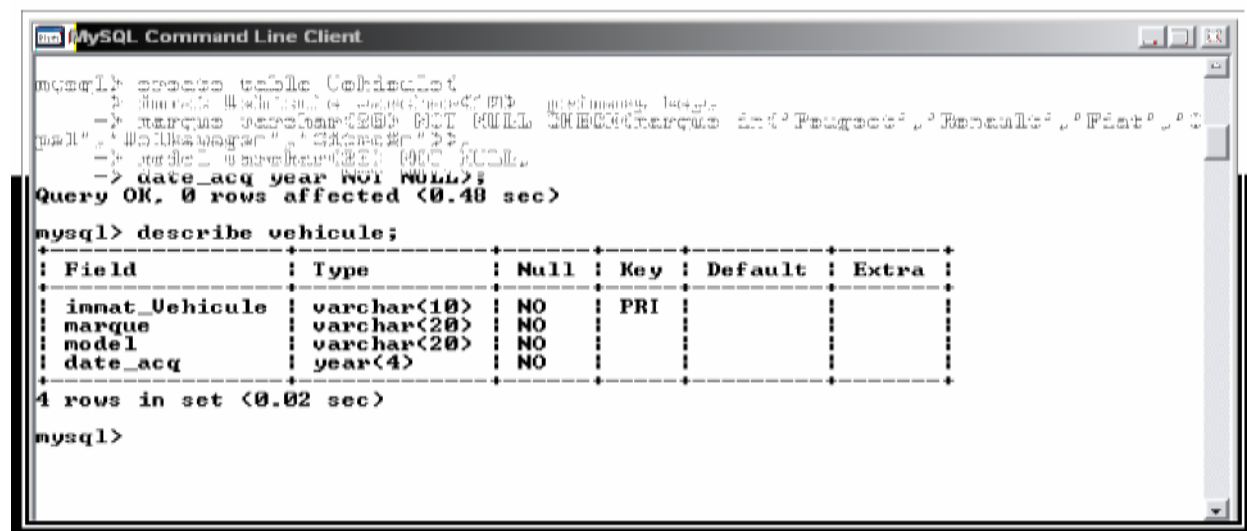
NB: Pour attribuer la date d'aujourd'hui comme valeur par défaut, il faut utiliser le type "timestamp" sinon ça ne marche pas avec le type date.

Illustrations graphiques de la création de différentes tables:

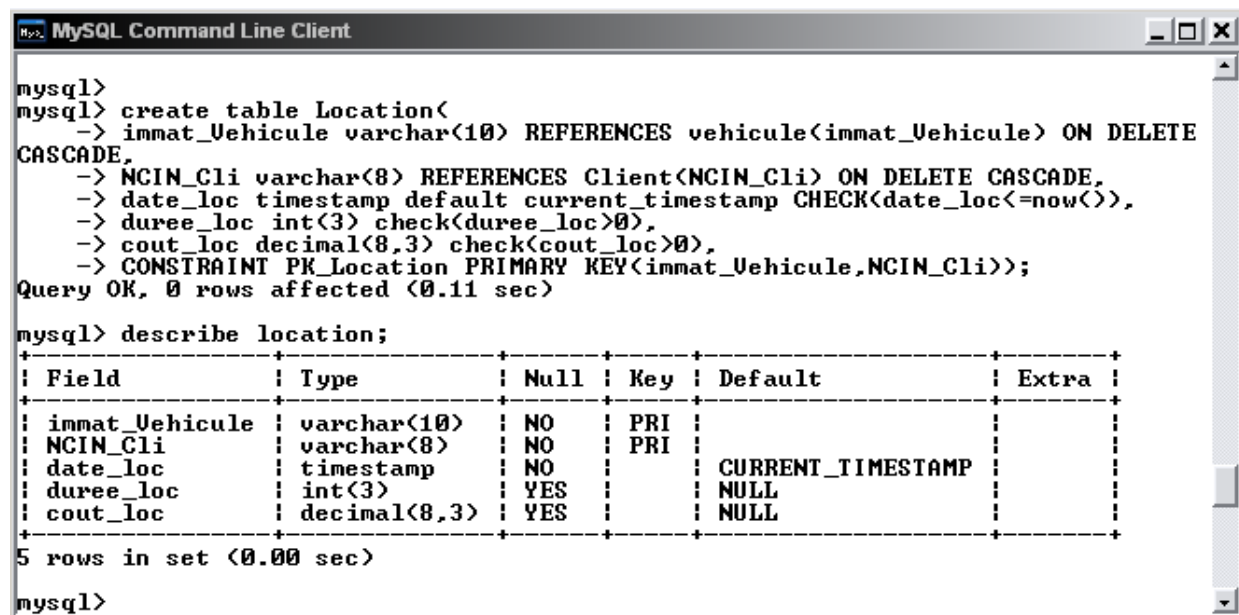
Table Client



❖ Table Vehicle



❖ Table Location



I.3 Modification de la structure d'une table

❖ Syntaxe de modification de la structure d'une table :

```
ALTER TABLE nom-table
( ADD [COLUMN] (nom-col type-col [DEFAULT valeur] [contrainte-col])* ,
  | ADD [CONSTRAINT] (contrainte_table)* ,
  | MODIFY [COLUMN] nom-col [type-col] [DEFAULT valeur] [contrainte-col] ,
  | MODIFY [CONSTRAINT] nom_contrainte
  | DROP COLUMN nom-col ,
  | DROP {PRIMARY KEY
  | CONSTRAINT nom_contrainte }
  | RENAME TO <nouveau_nom_table>
);
```

❖ Syntaxe de suppression d'une table :

```
DROP TABLE nom_table;
```

a) Ajout de colonnes à une table

Ajouter à la table **Client** un champ nommé **Tel_Cli** de type texte et de taille égale à 10.

```
ALTER TABLE Client
ADD COLUMN Tel_Cli varchar(10);
```

N.B : l'utilisation de () après le mot clé "COLUMN" est facultatif quand il s'agit de l'ajout d'une seule colonne.

b) Modification d'une colonne de la table

Modifier le type de donnée de la colonne "**Tel_Ci**" de la table "**Client**" en numérique de 8 chiffres.

```
ALTER TABLE Client
MODIFY COLUMN Tel_Cli INT(8);
```

c) Suppression des colonnes d'une table

Supprimer la colonne "**Adresse_Cli**" dans la table "**Client**".

```
ALTER TABLE Client
DROP COLUMN Adresse_Cli;
```

d) Ajout d'une contrainte

Vérifier que les valeurs de "**Annee_Acq**" dans la table "**Vehicule**" est entre 2020 et 2026.

```
ALTER TABLE Vehicule
ADD CONSTRAINT CHECK Annee_Acq BETWEEN 2020 AND 2026;
```

Remarque: Si on veut attribuer la ville "tunis" comme valeur par défaut à tous les clients, il ne s'agit pas d'ajouter une contrainte mais de modifier la colonne "**Ville**"

```
ALTER TABLE Client
MODIFY COLUMN Ville VARCHAR(20) DEFAULT 'tunis';
```

e) Modification de la clé primaire d'une table

Modifier la clé primaire actuelle de la table Location par la clé primaire composée suivante:

ImmatVehicule, NCIN, Date Loc

```
ALTER TABLE Location
DROP PRIMARY KEY
ADD PRIMARY KEY (immat_Vehicule,NCIN_Cli,date_loc);
```

f) Suppression d'une contrainte

Supprimer la clé primaire de la table "Client"

```
ALTERTABLEClient
DROP PRIMARY KEY;
```

g) Activer/désactiver une contrainte

Désactiver la contrainte de clé primaire de la table "client"

```
ALTERTABLEClient
DISABLE KEYS;
```

Réactiver cette même contrainte

```
ALTERTABLEClient
ENABLE KEYS;
```

h) Renommer une table

Renommer la table "Vehicule" en "Voiture"

```
ALTERTABLEVehicule
RENAME TOVoiture;
```

i) Suppression d'une table

Supprimer la table "Client"

```
DROP TABLE Client;
```

j) Suppression de la base de données

Supprimer la base de données "Gestion_Location"

```
DROP DATABASE Gestion_Location;
```

I) Introduction

L'opération de mise à jour des tables d'une base de données regroupe :

- la commande d'insertion,
- la commande de modification
- la commande de suppression

II) Mise à jour de données**II.1.1 Insertion de lignes**

Syntaxe : Insert Into Nom de table (Liste_Nom_Colonne)**Values** (Liste_Valeur) ;

- Liste_Nom_Colonne : sert à préciser la liste des colonnes dans lesquelles va s'effectuer l'opération d'insertion.

Remarques :

- la liste des colonnes peut être omise dans le cas où l'opération d'insertion concerne toutes les colonnes de la table.

Exemple :**Insert into Clients****Values ('A301','Tounsi','Weal','Tunisienne') ;**

Equivalente à

Insert into Clients (COD_CLI, Nom_Cli, Prn_Cli, Nat_Cli)**Values ('A301','Tounsi','Weal','Tunisienne') ;**

Equivalente à

Insert into Clients (Nom_Cli , Nat_Cli, COD_CLI, Prn_Cli)**Values ('Tounsi', 'Tunisienne', 'A301','Weal') ;**

- Les types de données de la liste_Valeur doivent être compatibles avec ceux de colonnes de la table.
- Unicité de ligne (contrainte de **clé primaire**)
- Valeurs obligatoires associées à une colonne (clause Not Null)
- Contrainte d'intégrité référentielle (cl é étrangère)
- Vérification de la condition de validité **CHECK**)

II.1.2 Suppression de lignes :**Syntaxe : Delete** From nom table**Where** condition ;

- la clause Where est optionnelle, si elle est absente alors toutes les lignes de la table concernée seront supprimées.

ExempleDelete from clients  supprime toutes les lignes de la table clients

- La clause **Where** sert à indiquer la condition à vérifier pour les lignes à supprimer.

Exemple

- Si la commande **Delete** ne respecte pas la contrainte d'intégrité référentielle, elle ne peut pas être exécutée.

Exemple1

- Supprimer le fournisseur numéro 707

- Delete from Fournisseur

Where NUm_Frs=707 ;

Le client numéro 707 sera supprimé car il n'existe pas dans la table Articles.

Exemple2

- Supprimer le fournisseur numéro 714
- Delete from Fournisseur
Where NUm_Frs = 714 ;

 La suppression du fournisseur numéro 714 ne peut pas être réalisée car ce numéro se trouve dans la table Articles (num_frs est une clé étrangère dans la table Articles) et on n'a pas utilisé la clause « ON DELETE CASCADE » lors de la définition du clé étrangère Num_frs dans la table Articles

create table articles

**(num_art integer primary key,
des_art varchar(20) not null,
col_art varchar(20),**

```

pv_art float check(pv_art>0),
pa_art float check(pa_art>0),
num_frs integer references fournisseurs (num_frs));

```

- ✓ La suppression d'une ligne appartenant à une table donnée peut entraîner la suppression d'autres lignes appartenant à d'autres tables lorsqu'il existe des contraintes d'intégrité référentielle de suppression en cascade (utilisation de la clause **On delete Cascade** dans la définition des clés étrangères).

Exemple

- Supprimer le client ayant le code 'A355'
- Delete from Clients
Where Cod_Cli = 'A355' ;

 On remarque que les ventes effectuées par ce client ont été supprimées (toute ligne de la table Ventes ayant Cod_Cli='A355' seront supprimées automatiquement)

II.1.3 Modification de lignes

Syntaxe : Update nom de table

Set Nom_Colonne1= Expression1 [, Nom_Colonne2= Expression2..]

Where condition ;

- La modification peut toucher une ou plusieurs colonnes.
- Si on ne spécifie aucune condition dans la clause **Where**, l'update sera réalisé sur toutes les lignes de la table.
- Les principaux cas pour lesquelles l'ordre **Update** ne peut aboutir sont les mêmes que ceux qui ont été mentionnées pour l'ordre **Insert**.

Exemple :

- * Mettre à jour le prix de vente de l'article numéro 501 de valeur 18.75

Update Articles

Set pv_art = 18.75

Where num_Art = 501 ;

- Mettre à jour l'identité du fournisseur numéro 714 (Nom : « Tounsi » Tel : 98970020)

Update Fournisseurs

Set Nom_Frs = 'Tounsi', tel_frs=989790020

Where num_frs = 714 ;

- Transformer les nationalités des clients en majuscules.

Update clients

Set nat_cli=upper(nat_cli);

- Augmenter le prix de ventes des articles de 18% par apport au prix initial.

Update articles

Set pv_art=(pv_art*118)/100;

III) Recherche de données : requêtes

III.1 Principe

La recherche de données est une opération qui consiste en la recherche de données appartenant à la base.

Une recherche peut s'effectuer en :

- ♦ Une projection sur certaines colonnes d'une table,
- ♦ Une sélection sur certaines lignes d'une table,
- ♦ Une jointure sur deux tables,
- ♦ Toute combinaison de projection, sélection et jointure,

La commande permettant d'effectuer une recherche dans une table de donnée est la commande SELECT.

III.2 La commande **SELECT** :

<i>Syntaxe :</i> SELECT Nom_champ1, Nom_champ2,... FROM Nom_Table ; La.....consiste à effectuer un sous ensemble de colonnes, d'une table. - Elle ne concerne qu'une seule table. - Toutes les lignes relatives aux colonnes feront parti du résultat. Projection	<i>Syntaxe :</i> SELECT Nom_champ1, Nom_champ2,... FROM Nom_Table WHERE Condition ; la clause WHERE permet d'introduire une condition pour spécifier les lignes à afficher. - La.....consiste à afficher un sous ensemble de lignes d'une table. Sélection
NB : La sélection est souvent combinée avec la projection. Dans ce cas le résultat est un sous ensemble de lignes et de colonnes de cette table.	

• Select *

From Nom_Table ;

Cette commande permet d'afficher toutes les colonnes de la table.

Exemple : Afficher la liste des clients

Select * From clients ;

III.2.1) La clause **DISTINCT**:

Rôle: n'afficher qu'un exemplaire de chaque jeu de lignes identiques.

Afficher les nationalités des clients:

Remarque:

- Pour construire une condition, on dispose d plusieurs opérateurs: =, >=, <=, >, <, <> et ! qui est la négation soit !=, !<, !>.
- Pour combiner plusieurs conditions, utiliser les opérateurs logiques AND, OR et NOT
- Pour comparer un champ à du texte, on utilisera le caractère « ' » pour encadrer le texte. Pour comparer un champ à une valeur numérique, ce caractère ne doit pas être employé.

III.2.2 L'utilisation des attributs **Between, In, Like**:

Ces attributs permettent de spécifier les conditions de recherche.

- ❖ **Between** : peut être utilisé si une valeur (caractère, date, numérique) se situe entre 2 valeurs.

Exemple : la liste des articles vendus dans la période de 01/01/2007 jusqu'au 8/01/2007.

- ❖ **IN** : vérifie si une valeur de colonne correspond à l'une de celle spécifiée dans la liste.
- ❖ **NOT IN** : vérifie si une valeur de colonne ne correspond pas à l'une de celle spécifiée dans la liste.

Exemple : la liste des magasins de Sfax et gabes.

Select * From magasins

Where upper (loc_mag) in ('SFAX','GABES');

```

Select *
From magasins
Where upper (loc_mag) = 'SFAX' or upper (loc_mag) = 'GABES';

```

Exemple : la liste des magasins autres que de Sfax et gabes.

```

Select *
From magasins
Where upper(loc_mag) not in ('SFAX','GABES');

```

Exemple : la liste des clients tunisiens et algériens

```

Select *
From clients
Where upper (nat_cli) in ('TUNISIEN','ALGERIEN');

```

COD_	NOM_CLI	PRN_CLI	NAT_CLI
A301	NAFTI	SAMIR	TUNISIEN
A360	BEN BRAHIM	HOTHMAN	ALGERIEN
A355	TRABELSI	ABIR	TUNISIEN
A390	LOUKIL	CHOKRI	TUNISIEN
A300	LAJDEL	AYA	TUNISIEN
A377	LAJNEF	KAMEL	TUNISIEN

LIKE : sélectionne les lignes en comparant la valeur d'une colonne de type caractère à une chaîne de caractère.

Remarque : le caractère % représente n'importe quel nombre de caractère.

Le caractère _ représente un seul caractère quelconque.

Exemple : la liste des articles commençant par « Ch »

```

Select *
From articles
Where des_art like 'Ch%';

```

Exemple : la liste des articles commençant par « Ch » et dont le prix de vente est > 70 d

```

Select *
From articles
Where des_art like 'Ch%' and pv_art>70;

```

Exemple : la liste des fournisseurs dont les noms commençant par « Ka » et de taille 6

```

Select *
From fournisseurs
Where nom_frs like 'KA ';

```

III.2.3 L'utilisation d'expressions

Exemple : Calculer la marge bénéficiaire de chaque article (pv_art – pa_art)

```

Select pv_art-pa_art "marge bénéficiaire"
From articles;

```

Exemple : afficher le prénom et le nom de chaque client séparé par un espace.

```

Select prn_cli||' '||nom_cli "NOM PRENOM"
From clients;

```

III.2.4 L'utilisation de fonctions de calcul (fonctions agrégats)

- SQL offre certaines fonctions standard de calcul appelées fonctions agrégats. Ces fonctions ne peuvent être utilisées qu'avec la commande **SELECT** et en dehors de la clause **WHERE**.

- ♦ **COUNT () :** compte le nombre de ligne
- ♦ **SUM () :** additionne les valeurs d'une colonne numérique

- ♦ **MIN ()** : trouve la valeur minimale d'une colonne de type caractère ou numérique.
- ♦ **MAX ()** : trouve la valeur maximale d'une colonne de type caractère ou numérique.
- ♦ **AVG ()** : calcule la moyenne des valeurs d'une colonne.

Nombre des fournisseurs

```
Select count (*) "Nombre de Fournisseurs"  
From Fournisseurs;
```

☐ Nombre des clients tunisiens

```
Select count (*) "Nombre des clients Tunisiens"  
From clients  
Where upper (nat_cli) ='TUNISIEN';
```

Nombre des clients étrangers :

```
Select count (*) "Nombre des clients étrangers"  
From clients  
Where upper (nat_cli) <>'TUNISIEN';
```

☐ Nombre des articles fournis par le fournisseur numéro 777

```
Select count (*)  
From Articles  
Where num_frs=777;  
La somme des prix d'achat des articles  
Select sum(pa_art)  
From articles;
```

☐ La moyenne des prix d'achat des articles

```
Select avg (pa_art)  
From articles
```

```
Select sum(pa_art)/count(*)  
From articles;
```

☐ L'article le prix plus cher (ayant le prix de vente le plus élevé)

```
Select max (pv_art)  
From articles;
```

☐ La désignation de l'article le prix plus cher (ayant le prix de vente le plus élevé)

```
Select des_art "Article plus cher"  
From articles  
Where pv_art=(select max(pv_art)  
From articles);
```

L'article le prix moins cher (ayant le prix de vente le plus bas)

```
Select min (pv_art) "Prix bas"  
From articles;
```

☐ Le numéro de l'article le moins cher (ayant le prix de vente le plus cher)

```
Select num_art  
From articles  
Where pv_art= (select min(pv_art) From articles);
```

☐ Le numéro de fournisseur qui fourni l'article le plus cher (ayant le prix d'achat le plus cher).

```
Select num_frs  
From articles  
Where pa_art= (select max (pa_art) From articles);
```

☐ Le nombre des clients qui ont acheté des articles dans la période du 01/01/2026 jusqu'à 08/01/2026.

Select count (cod_cli)

From ventes Where dat_vnt between;

☐ Trouver les désignations des articles dont le prix de vente est supérieur au prix de vente moyen.

Select des_art

From articles Where pv_art > (select avg (pv_art) From articles);

Trouver les informations relatives aux fournisseurs qui n'ont pas fourni des produits (nom et téléphone).

Select nom_frs, tel_frs

From fournisseurs

Where num_frs not in (select num_frs From articles);

☐ Trouver les produits (désignation et couleur) qui ne sont pas encore vendus.

Select des_art, col_art

From articles

Where num_art not in (select num_art From ventes);

III.2.5 Tri des données

La clause **ORDER BY** spécifie la liste des colonnes sur lesquelles l'affichage doit être trié (**ASC** (ascendant) ou **DESC** (descendant)).

- ***C'est le tri par ordre croissant qui est pris par défaut.***

☐ Liste des clients triés par ordre croissant des noms

Select *

From clients

Order by nom_cli;

☐ Liste des clients triés par ordre décroissant des prénoms

Select *

From clients

Order by prn_cli desc ;

☐ Liste des articles triés par ordre décroissant des prix d'achat et par ordre croissant des prix de vente

Select *

From articles

Order by pa_art desc , pv_art;